Lab05

#cyber-security

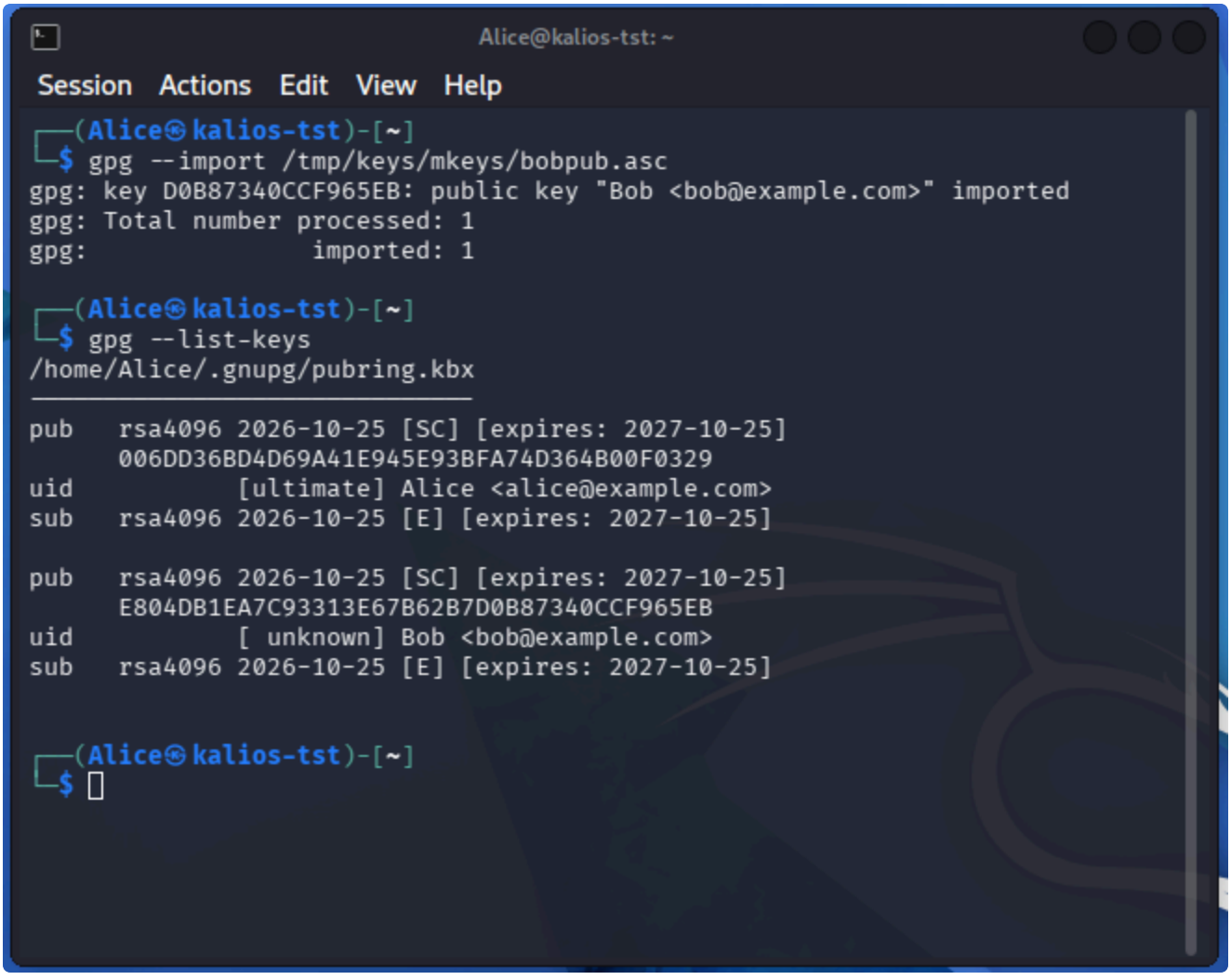
Setup:

We make 3 new users to simulate this scenario, those being Alice and Bob (the communicating party) and Mallory (the one who places themselves in the middle of their communication). Afterwards, we generate key value pairs for all users and export them all into the a `tmp/keys` folder for convenience.

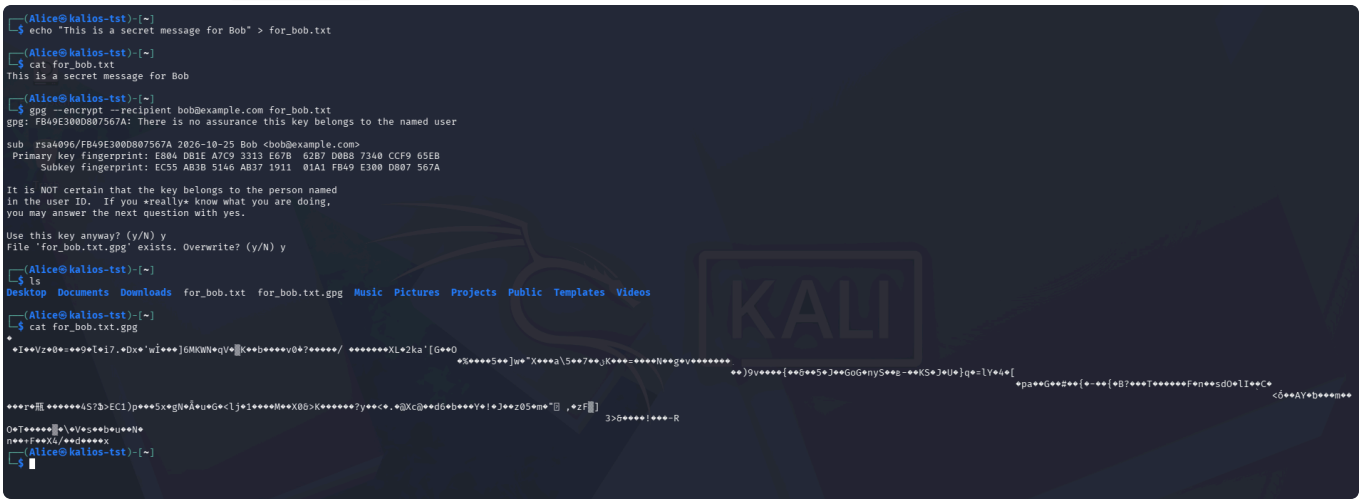
Since Mallory is the one intercepting the conversation, she will export her public key with the same name as Bob's public key, we will put it within a subdirectory of the `keys` folder for convenience.

We should end up with a `keys` folder containing the following (Alice and Bob's public keys as well as Mallory's public key faked as bobs public key)

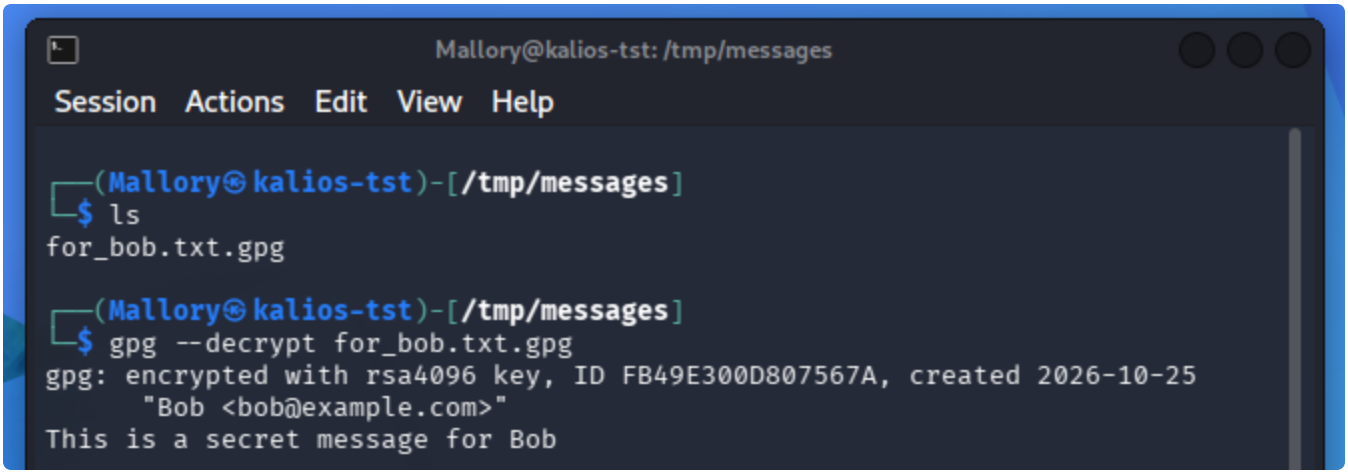
1 - Alice imports Mallory's faked key:



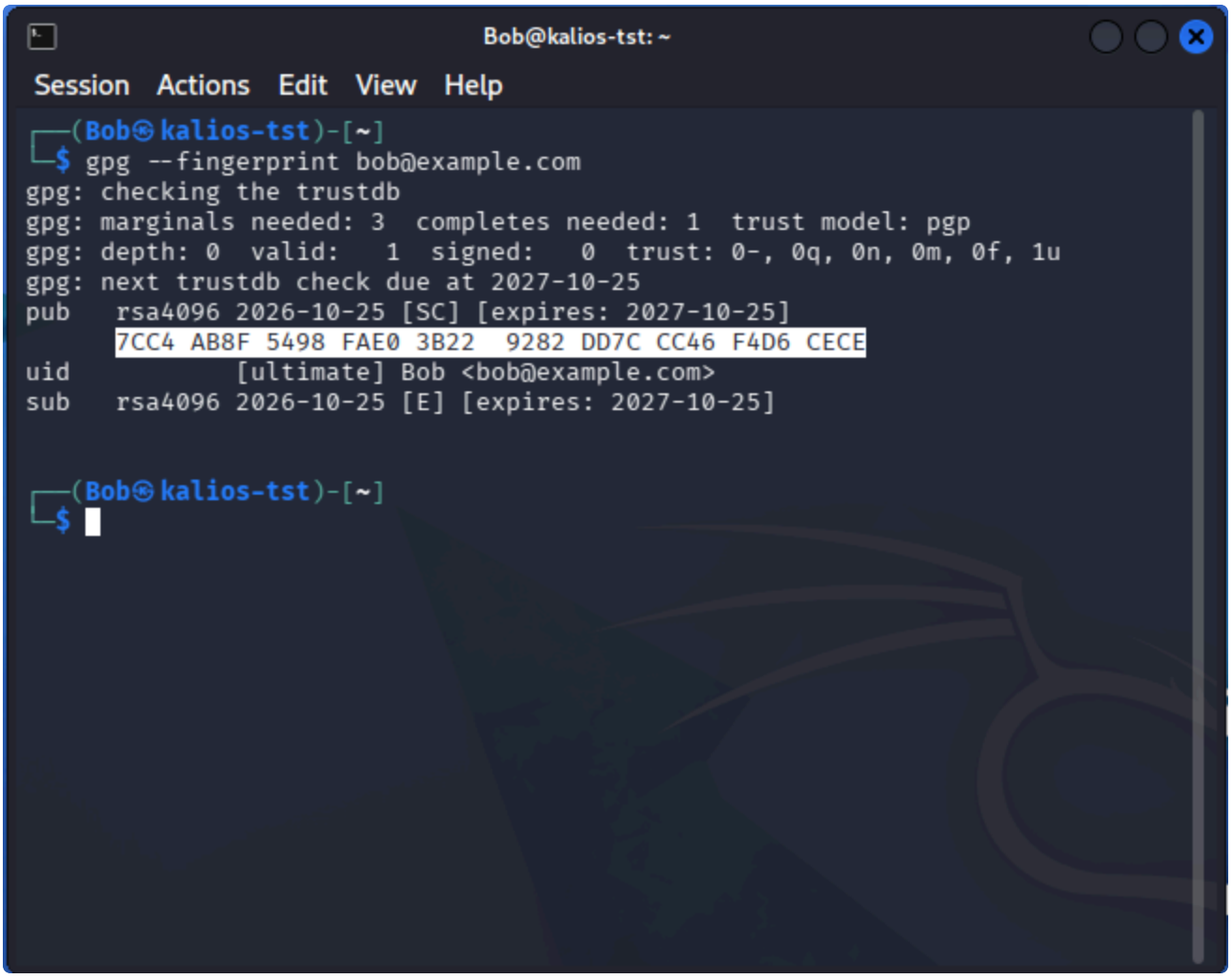
Alice then makes a message for Bob and encrypts it using what she believes is Bob's public key.
(for this we first create a sample file - we also them move it to a subfolder within the temp directory called `messages` which we also create for convenience).



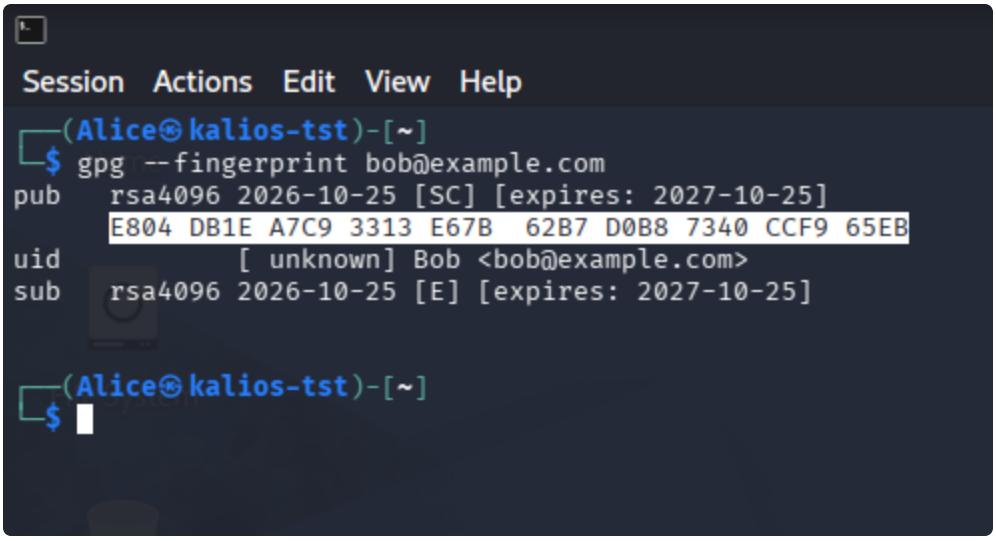
2 - Mallory Reads the Confidential Message:



3 - Bob checks his fingerprint and sends it to Alice via a third channel:

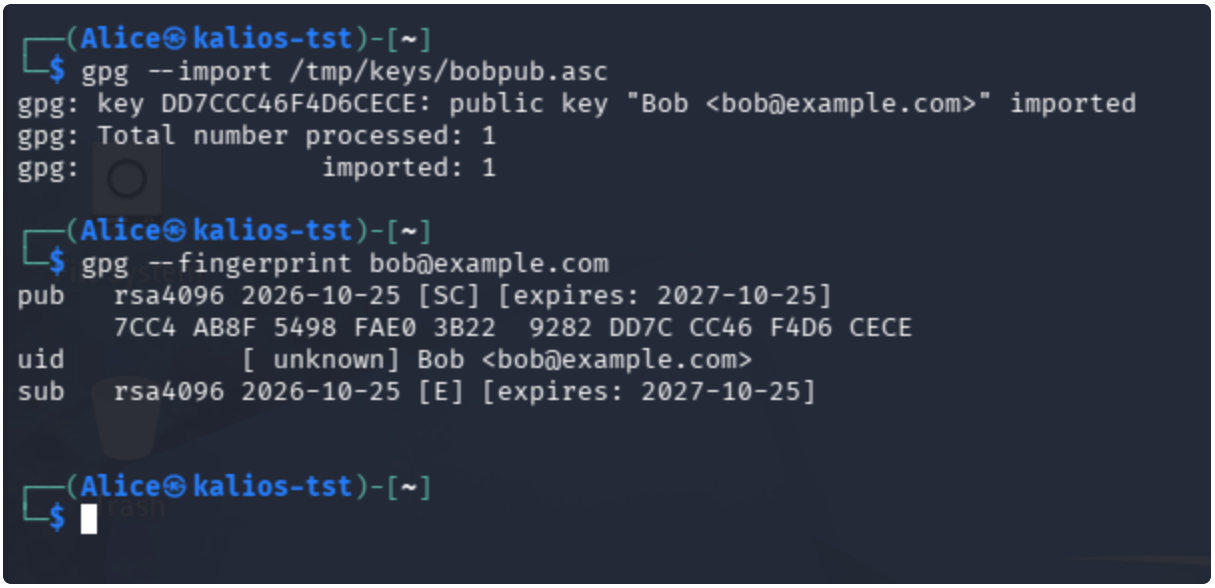


Alice can then check the validity of the key she posseses:



As we can see by the output, the two fingerprints differ.

After importing the proper key, we can verify it via checking the fingerprint



Then we can set this key as a trusted key within the web of trust:

```
┌──(Alice㉿kalios-tst)-[~]
└─$ gpg --edit-key bob@example.com
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.


pub  rsa4096/DD7CCC46F4D6CECE
     created: 2026-10-25  expires: 2027-10-25  usage: SC
     trust: unknown       validity: unknown
sub  rsa4096/F94A94ED173BC425
     created: 2026-10-25  expires: 2027-10-25  usage: E
[ unknown] (1). Bob <bob@example.com>

gpg> trust
pub  rsa4096/DD7CCC46F4D6CECE
     created: 2026-10-25  expires: 2027-10-25  usage: SC
     trust: unknown       validity: unknown
sub  rsa4096/F94A94ED173BC425
     created: 2026-10-25  expires: 2027-10-25  usage: E
[ unknown] (1). Bob <bob@example.com>

Please decide how far you trust this user to correctly verify other users' keys
(by looking at passports, checking fingerprints from different sources, etc.)

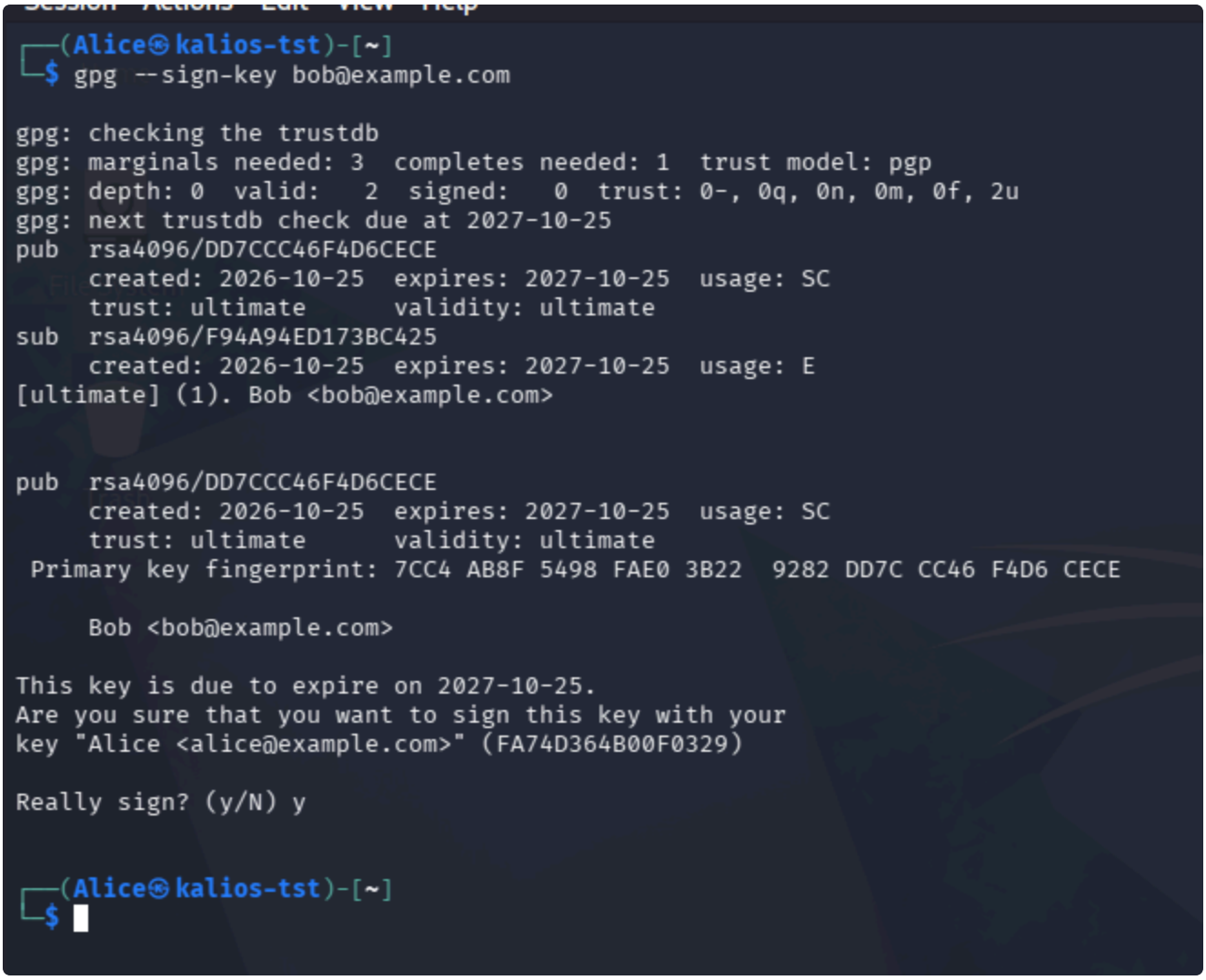
  1 = I don't know or won't say
  2 = I do NOT trust
  3 = I trust marginally
  4 = I trust fully
  5 = I trust ultimately
  m = back to the main menu

Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y

pub  rsa4096/DD7CCC46F4D6CECE
     created: 2026-10-25  expires: 2027-10-25  usage: SC
     trust: ultimate      validity: unknown
sub  rsa4096/F94A94ED173BC425
     created: 2026-10-25  expires: 2027-10-25  usage: E
[ unknown] (1). Bob <bob@example.com>
Please note that the shown key validity is not necessarily correct
unless you restart the program.

gpg>
```

We can solidify the trust regarding this particular key even further by signing it using our private key.



Closing Thoughts:

1. Why doesn't GPG detect MITM attacks automatically? - GPG does provide an automatic warning regarding keys that are not trusted within the web of trust model. However without that indirect trust the system cannot compare against anything that would invalidate the key.
2. What is a fingerprint and why is it important? - It's a hash of the public key used to check for integrity
3. Why is email not a secure channel for exchanging keys? - Email is not end-to-end encrypted, allowing for potential interceptions at each stop the key exchange travels through.